RAG

**PROJET DE FIN D'ANNÉE (PFA)**

2025 – 2026

# Cahier des Charges

## Système RAG Souverain pour l'Entreprise

Infrastructure d'Automatisation pour l'Extraction,
l'Indexation et l'Interrogation de Données d'Entreprise
via un LLM Open-Source Local


**Auteur :** [Votre Nom]

**Encadrant :** [Nom de l'encadrant]

**Établissement :** [Nom de l'école / université]



**Date :** Juillet 2025

**Version :** 1.0

# Résumé Exécutif

**Synthèse du projet**

Ce projet de fin d'année porte sur la conception et le développement d'un système **RAG (Retrieval-Augmented Generation) souverain**, capable d'ingérer des documents d'entreprise hétérogènes (PDF, textes, bases SQL), de les indexer dans une base vectorielle locale, et de permettre l'interrogation en langage naturel via un **modèle de langage open-source** déployé entièrement sur l'infrastructure de l'entreprise, sans aucune transmission de données vers des services tiers.

**Contrainte principale**

**Souveraineté totale des données** : aucune donnée confidentielle ne doit quitter l'infrastructure de l'entreprise à aucune étape du pipeline.

Le projet se découpe en trois défis majeurs :

1. **Défi Data** — Centraliser et découper intelligemment des données hétérogènes (PDF, SQL, textes libres) via un pipeline d'ingestion automatisé.
2. **Défi IA** — Construire un moteur RAG fiable avec un LLM local open-source (Mistral 7B via Ollama) qui ancre ses réponses sur les sources indexées.
3. **Défi Sécurité/Ops** — Garantir la confidentialité par conteneurisation complète (Docker Compose) sans dépendance réseau externe.

# Table des matières

**Résumé Exécutif** **1**

**1 Présentation du Projet** **4**
1.1 Contexte général . . . . . 4
1.2 Problématique . . . . . 4
1.3 Objectifs du projet . . . . . 4
1.4 Périmètre du projet . . . . . 4
1.4.1 Dans le périmètre (In Scope) . . . . . 4
1.4.2 Hors périmètre (Out of Scope) . . . . . 5

**2 Architecture Technique** **6**
2.1 Vue d'ensemble de l'architecture . . . . . 6
2.2 Stack technologique . . . . . 6
2.3 Architecture de conteneurisation . . . . . 6

**3 Spécifications Fonctionnelles** **9**
3.1 Module 1 — Ingestion et Parsing de Documents . . . . . 9
3.1.1 Description . . . . . 9
3.1.2 Fonctionnalités . . . . . 9
3.1.3 Entrées / Sorties . . . . . 9
3.2 Module 2 — Chunking Adaptatif . . . . . 10
3.2.1 Description . . . . . 10
3.2.2 Stratégies de chunking . . . . . 10
3.2.3 Fonctionnalités . . . . . 10
3.3 Module 3 — Vectorisation et Indexation . . . . . 10
3.3.1 Description . . . . . 10
3.3.2 Fonctionnalités . . . . . 11
3.3.3 Modèles d'embeddings supportés . . . . . 11
3.4 Module 4 — Retrieval (Recherche Sémantique) . . . . . 11
3.4.1 Description . . . . . 11
3.4.2 Fonctionnalités . . . . . 12
3.5 Module 5 — Génération (LLM Local) . . . . . 12

3.5.1 Description . . . 12
3.5.2 Structure du prompt système . . . 12
3.5.3 Fonctionnalités . . . 13
3.6 Module 6 — Interface Utilisateur (Streamlit) . . . 13
3.6.1 Fonctionnalités . . . 13
3.7 Module 7 — API REST (FastAPI) . . . 13
3.7.1 Endpoints principaux . . . 14

**4 Spécifications Non Fonctionnelles 15**
4.1 Performances . . . 15
4.2 Sécurité et Souveraineté . . . 15
4.3 Reproductibilité et Déploiement . . . 15
4.4 Maintenabilité . . . 16

**5 Plan de Développement 17**
5.1 Découpage en sprints (8 semaines) . . . 17
5.2 Structure du projet (arborescence) . . . 17

**6 Évaluation et Métriques 19**
6.1 Framework d'évaluation RAGAS . . . 19
6.2 Dataset de test . . . 19

**7 Risques et Mitigations 20**

**8 Livrables Attendus 21**

**Glossaire 22**

**Signatures et Validation 23**

# Chapitre 1

# Présentation du Projet

## 1.1 Contexte général

Les entreprises accumulent un volume croissant de données non structurées : rapports PDF, documentations internes, bases de données relationnelles, notes techniques. L'exploitation de cette connaissance reste difficile et chronophage par les méthodes traditionnelles de recherche par mots-clés.

L'émergence des **Large Language Models (LLM)** et de la technique **RAG (Retrieval-Augmented Generation)** offre une solution : permettre aux utilisateurs d'interroger leur corpus documentaire en langage naturel, avec des réponses précises ancrées sur les sources.

Cependant, l'adoption de solutions cloud (OpenAI, Anthropic, Google) soulève des risques critiques de **fuite de données confidentielles**. Ce projet propose une alternative 100% locale et souveraine.

## 1.2 Problématique

**Problématique centrale**

*Comment concevoir et déployer un pipeline RAG souverain, capable d'ingérer des documents d'entreprise hétérogènes, d'en extraire la connaissance via un LLM open-source local, et de garantir la confidentialité des données par conteneurisation — le tout sur une infrastructure reproductible en environnement isolé ?*

## 1.3 Objectifs du projet

## 1.4 Périmètre du projet

### 1.4.1 Dans le périmètre (In Scope)

— Pipeline d'ingestion : PDF (natif et scanné), fichiers TXT/Markdown, tables SQL

— Chunking adaptatif selon la structure du document

— Génération d'embeddings locaux avec `sentence-transformers`

— Stockage vectoriel avec ChromaDB ou Qdrant

| N° | Objectif | Priorité |
|---|---|---|
| O1 | Construire un pipeline d'ingestion automatique de documents PDF, TXT et SQL | **Critique** |
| O2 | Implémenter un chunker adaptatif selon le type de document | **Critique** |
| O3 | Vectoriser les chunks et les stocker dans une base vectorielle locale | **Critique** |
| O4 | Intégrer un LLM local (Mistral 7B via Ollama) comme moteur de génération | **Critique** |
| O5 | Développer une interface utilisateur simple (Streamlit) | **Important** |
| O6 | Évaluer les performances du système avec des métriques RAGAS | **Important** |
| O7 | Conteneuriser l'ensemble du système via Docker Compose | **Important** |
| O8 | Exposer une API REST (FastAPI) pour l'intégration avec des tiers internes | **Souhaitable** |

TABLE 1.1 – Objectifs du projet par priorité

— Génération de réponses avec Mistral 7B via Ollama

— Interface Streamlit pour la démonstration

— Déploiement complet Docker Compose

— Évaluation avec framework RAGAS

### 1.4.2 Hors périmètre (Out of Scope)

— Authentification et gestion des utilisateurs

— Traitement de fichiers audio ou vidéo

— Fine-tuning du modèle LLM

— Déploiement en production sur serveur distant

— Application mobile

— Scalabilité multi-nœuds (Kubernetes)

# Chapitre 2

# Architecture Technique

## 2.1 Vue d'ensemble de l'architecture

Le système est composé de deux pipelines distincts mais interconnectés :

1. **Pipeline d'ingestion (offline)** — traitement des documents et construction de l'index vectoriel
2. **Pipeline de requête (online)** — traitement des questions utilisateur et génération de réponses

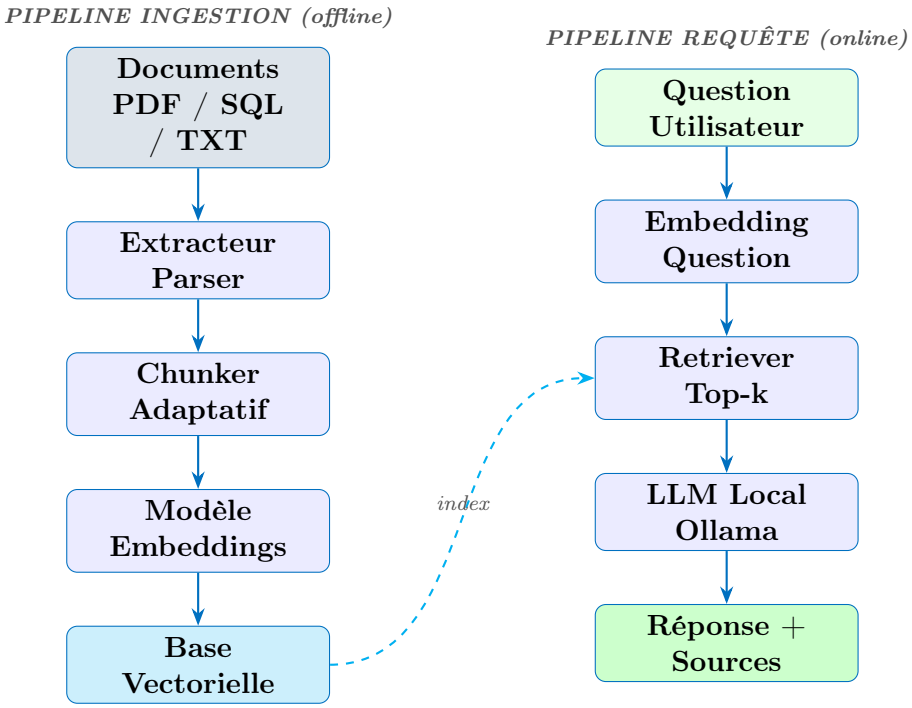



FIGURE 2.1 – Architecture générale du système RAG souverain

## 2.2 Stack technologique

## 2.3 Architecture de conteneurisation

Listing 2.1 – Structure Docker Compose (services)

```
services:
```

| Couche | Technologie choisie | Justification |
|---|---|---|
| LLM local | Ollama + Mistral 7B Instruct | Excellent FR/EN, tourne sur CPU/GPU, licence Apache 2.0 |
| Embeddings | sentence-transformers | 100% local, modèles multilingues disponibles |
| Base vectorielle | ChromaDB (puis Qdrant) | Zero-config en Python, Docker-natif pour la prod |
| Orchestration RAG | LlamaIndex | Pipeline d'ingestion documentaire natif, chunking intégré |
| Parsing PDF | pdfplumber / PyMuPDF | Extraction texte + structure, gestion des tableaux |
| OCR (PDF scan) | Tesseract + pdf2image | Open-source, multilingue, intégration Python simple |
| Interface | Streamlit | Déployable en 50 lignes, idéal pour démo jury |
| API | FastAPI | REST async, documentation auto Swagger |
| Conteneurisation | Docker Compose | Isolation totale, reproductibilité |
| Évaluation | RAGAS | Framework dédié à l'évaluation de pipelines RAG |

TABLE 2.1 – Stack technologique du projet

```
  ollama:           # LLM local (Mistral 7B)
    image: ollama/ollama
    ports: ["11434:11434"]
    volumes: ["./models:/root/.ollama"]
    networks: [rag_network]

  chromadb:         # Base vectorielle
    image: chromadb/chroma
    ports: ["8000:8000"]
    volumes: ["./chroma_data:/chroma/chroma"]
    networks: [rag_network]

  app:              # Application principale
    build: ./app
    depends_on: [ollama, chromadb]
    ports: ["8501:8501"]   # Streamlit
    volumes: ["./data:/app/data"]
    networks: [rag_network]

```

```
networks:
  rag_network:
    driver: bridge
    internal: true    # Isolation reseau totale
```

La clé de souveraineté : le réseau Docker est défini avec `internal: true`, ce qui **interdit toute connexion sortante** vers Internet.

# Chapitre 3

# Spécifications Fonctionnelles

## 3.1 Module 1 — Ingestion et Parsing de Documents

### 3.1.1 Description

Ce module constitue le point d'entrée du pipeline. Il est responsable de détecter le type de chaque fichier source et d'appliquer la stratégie d'extraction appropriée pour produire du texte propre et structuré.

### 3.1.2 Fonctionnalités

**Fonctionnalités — Ingestion**

- **F1.1** — Détection automatique du type de fichier (PDF natif, PDF scanné, TXT, MD, SQL)
- **F1.2** — Extraction du texte brut depuis les PDF natifs (pdfplumber / PyMuPDF)
- **F1.3** — OCR des PDF scannés via Tesseract + pdf2image
- **F1.4** — Lecture et nettoyage des fichiers TXT et Markdown
- **F1.5** — Connexion à une base SQL et conversion des enregistrements en phrases naturelles
- **F1.6** — Nettoyage du texte : suppression des headers/footers répétitifs, caractères parasites
- **F1.7** — Extraction et conservation des métadonnées (nom fichier, page, date, type)
- **F1.8** — Traitement par lot (batch) d'un répertoire complet de documents

### 3.1.3 Entrées / Sorties

| | **Description** |
|---|---|
| **Entrée** | Fichiers PDF, TXT, MD dans un répertoire `/data/documents/` ou connexion SQL |
| **Sortie** | Liste de documents Python avec champs `text`, `source`, `page`, `type` |

## 3.2 Module 2 — Chunking Adaptatif

### 3.2.1 Description

Ce module découpe le texte extrait en segments (chunks) de taille optimale. La stratégie de découpe est **adaptée automatiquement** au type de document détecté, ce qui constitue la principale contribution technique originale du projet.

### 3.2.2 Stratégies de chunking

| Type | Stratégie | Description | Overlap |
|---|---|---|---|
| PDF structuré | Par section | Découpe aux titres/sous-titres détectés | 0 |
| PDF non structuré | Fenêtre fixe | 512 tokens par chunk | 50 tokens |
| Texte libre | Fenêtre fixe | 512 tokens par chunk | 50 tokens |
| Données SQL | Par enregistrement | 1 ligne → 1 phrase naturelle | 0 |
| Tableaux | Par ligne | Header + valeurs de chaque ligne | 0 |

TABLE 3.1 – Stratégies de chunking par type de document

### 3.2.3 Fonctionnalités

**Fonctionnalités — Chunking**

- **F2.1** — Détection automatique du type de structure du document
- **F2.2** — Chunking par section pour les PDF avec titres détectables
- **F2.3** — Chunking par fenêtre glissante avec overlap paramétrable
- **F2.4** — Chunking par enregistrement pour les données tabulaires et SQL
- **F2.5** — Injection automatique des métadonnées dans chaque chunk
- **F2.6** — Filtrage des chunks trop courts ($< 50$ tokens) ou vides
- **F2.7** — Paramétrage via fichier de configuration YAML

## 3.3 Module 3 — Vectorisation et Indexation

### 3.3.1 Description

Ce module convertit chaque chunk textuel en vecteur numérique (embedding) via un modèle local, puis stocke ces vecteurs dans la base vectorielle pour une recherche ultérieure par similarité.

### 3.3.2 Fonctionnalités

**Fonctionnalités — Vectorisation**

- **F3.1** — Chargement et utilisation locale du modèle `sentence-transformers`
- **F3.2** — Génération d'embeddings pour chaque chunk (batch processing)
- **F3.3** — Stockage des vecteurs + texte + métadonnées dans ChromaDB
- **F3.4** — Détection et évitement des doublons lors de la ré-indexation
- **F3.5** — Persistance de l'index entre les redémarrages (volume Docker)
- **F3.6** — Affichage du nombre de chunks indexés et de la taille de l'index
- **F3.7** — Suppression et réindexation complète sur demande

### 3.3.3 Modèles d'embeddings supportés

| Modèle | Taille | Langue | Usage recommandé |
|---|---|---|---|
| `all-MiniLM-L6-v2` | 80 MB | EN | Corpus en anglais, machine limitée |
| `paraphrase-multilingual-MiniLM-L12-v2` | 420 MB | FR/EN/+ | Corpus mixte ou francophone |
| `nomic-embed-text` | 270 MB | EN | Haute performance anglais |

TABLE 3.2 – Modèles d'embeddings locaux disponibles

## 3.4 Module 4 — Retrieval (Recherche Sémantique)

### 3.4.1 Description

Lorsqu'un utilisateur pose une question, ce module la convertit en vecteur, puis recherche dans la base les $k$ chunks les plus proches sémantiquement pour les transmettre au LLM comme contexte.

### 3.4.2 Fonctionnalités

**Fonctionnalités — Retrieval**

- **F4.1** — Conversion de la question en vecteur (même modèle d'embedding)
- **F4.2** — Recherche par similarité cosinus dans la base vectorielle
- **F4.3** — Retour des top-k chunks avec leur score de similarité (k paramétrable, défaut : 4)
- **F4.4** — Filtrage par seuil de score minimum (éviter les réponses hors-sujet)
- **F4.5** — Filtrage par métadonnées (ex : chercher uniquement dans un fichier donné)
- **F4.6** — Retour des sources associées à chaque chunk récupéré

## 3.5 Module 5 — Génération (LLM Local)

### 3.5.1 Description

Ce module assemble le prompt final à partir de la question et des chunks récupérés, puis envoie ce prompt au LLM local via Ollama pour obtenir une réponse ancrée sur les sources.

### 3.5.2 Structure du prompt système

Listing 3.1 – Template de prompt RAG

```
SYSTEM:
Tu es un assistant d'entreprise. Tu reponds uniquement en te basant
sur les extraits de documents fournis ci-dessous. Si la reponse
n'est pas contenue dans les extraits, reponds clairement :
"Je ne trouve pas cette information dans les documents disponibles."
Ne fais aucune supposition. Cite toujours la source de ton information.

CONTEXTE:
[Extrait 1 - {source} - page {page}]
{texte_chunk_1}

[Extrait 2 - {source} - page {page}]
{texte_chunk_2}
```

```
14
15 QUESTION:
16 {question_utilisateur}
17
18 REPONSE:
```

### 3.5.3 Fonctionnalités

**Fonctionnalités — Génération LLM**

- **F5.1** — Envoi du prompt au LLM Mistral 7B via l'API Ollama locale
- **F5.2** — Streaming de la réponse pour un affichage progressif
- **F5.3** — Injection des sources dans le prompt pour ancrage factuel
- **F5.4** — Affichage des sources citées avec nom de fichier et numéro de page
- **F5.5** — Paramétrage de la température (défaut : 0.1 pour réponses déterministes)
- **F5.6** — Gestion des timeouts et erreurs de connexion à Ollama
- **F5.7** — Sélection du modèle LLM via configuration (Mistral, LLaMA, Phi-3...)

## 3.6 Module 6 — Interface Utilisateur (Streamlit)

### 3.6.1 Fonctionnalités

**Fonctionnalités — Interface**

- **F6.1** — Interface de chat en langage naturel (style messagerie)
- **F6.2** — Champ de téléversement de nouveaux documents (PDF, TXT)
- **F6.3** — Déclenchement manuel de l'indexation des documents déposés
- **F6.4** — Affichage de la réponse en streaming (mot par mot)
- **F6.5** — Affichage des sources utilisées sous chaque réponse
- **F6.6** — Affichage du score de confiance pour chaque source
- **F6.7** — Historique de la conversation dans la session
- **F6.8** — Indicateur de statut du système (LLM chargé, index prêt)

## 3.7 Module 7 — API REST (FastAPI)

### 3.7.1 Endpoints principaux

| Méthode | Endpoint | Description |
|---|---|---|
| POST | `/ingest` | Envoyer un fichier pour indexation |
| POST | `/query` | Poser une question, retourne réponse + sources |
| GET | `/status` | Statut du système (index, LLM, nombre de docs) |
| DELETE | `/index` | Supprimer et réinitialiser l'index vectoriel |
| GET | `/documents` | Lister les documents indexés |

TABLE 3.3 – Endpoints de l'API REST FastAPI

# Chapitre 4

# Spécifications Non Fonctionnelles

## 4.1 Performances

| Indicateur | Description | Cible |
|---|---|---|
| Latence de réponse | Temps entre envoi de la question et début de la réponse | $< 10$ s |
| Débit d'indexation | Nombre de pages PDF indexées par minute | $\geq 20$ pages/min |
| Précision retrieval | Recall@4 sur dataset de test | $\geq 0.75$ |
| Faithfulness | Score RAGAS de fidélité aux sources | $\geq 0.80$ |
| Disponibilité | Uptime du système pendant une session de démo | 100% |

TABLE 4.1 – Indicateurs de performance cibles

## 4.2 Sécurité et Souveraineté

**Exigences de souveraineté — Non négociables**

- **S1** — Aucune API externe ne doit être appelée (ni LLM, ni embeddings, ni autre service)
- **S2** — Le réseau Docker doit être configuré en mode `internal: true`
- **S3** — Tous les modèles (LLM + embeddings) doivent être téléchargés une fois puis utilisés hors ligne
- **S4** — Les données originales et l'index vectoriel sont stockés uniquement en local
- **S5** — Les logs système ne doivent pas contenir de contenu des documents indexés

## 4.3 Reproductibilité et Déploiement

- **R1** — Le système doit démarrer avec une seule commande : `docker compose up`
- **R2** — Un fichier `.env` centralise tous les paramètres configurables
- **R3** — Un fichier `README.md` documente les étapes d'installation complètes
- **R4** — Le code source doit être versionné sur Git avec historique de commits propre

## 4.4 Maintenabilité

— **M1** — Architecture modulaire : chaque composant est indépendant et testable isolément

— **M2** — Couverture de tests unitaires sur les fonctions critiques (chunker, retriever)

— **M3** — Documentation du code (docstrings Python) sur toutes les fonctions publiques

— **M4** — Fichier de configuration YAML pour les paramètres du pipeline

# Chapitre 5

# Plan de Développement

## 5.1 Découpage en sprints (8 semaines)

| Semaine | Phase | Tâches principales | Livrable |
|---|---|---|---|
| S1 | Cadrage et setup | Finalisation du CDC, choix définitif de la stack, setup environnement, dépôt Git | CDC signé + Repo initiali |
| S2 | Ingestion data | Parser PDF (natif + OCR), parser TXT, connecteur SQL, pipeline de nettoyage | Module parser fonctionnel |
| S3 | Chunking | Chunker adaptatif (4 stratégies), tests unitaires, gestion des métadonnées | Module chunker testé |
| S4 | Vectorisation | Intégration sentence-transformers, setup ChromaDB, pipeline d'indexation bout-en-bout | Index vectoriel peuplé |
| S5 | RAG complet | Intégration Ollama + Mistral, retriever, prompt engineering, réponse avec sources | Pipeline RAG fonctionnel |
| S6 | Interface | Interface Streamlit, API FastAPI, intégration des modules | Demo présentable |
| S7 | Évaluation | Dataset de test (30 Q/R), métriques RAGAS, rapport d'évaluation | Rapport de performance |
| S8 | Docker + rapport | Docker Compose complet, tests de souveraineté réseau, rédaction rapport final | Livrable final |

TABLE 5.1: Plan de développement sur 8 semaines

## 5.2 Structure du projet (arborescence)

Listing 5.1 – Arborescence du projet

```
rag-souverain/
```

```
        docker-compose.yml
        .env                          # Variables de
configuration
        README.md
        app/
              Dockerfile
              main.py                 # Point d'entree FastAPI
              streamlit_app.py        # Interface utilisateur
              config.yaml             # Configuration du
pipeline
              ingestion/
                    parser.py         # Extracteur multi-
format
                    chunker.py        # Chunker adaptatif
              indexing/
                    embedder.py       # Generation d'
embeddings
                    vector_store.py   # Interface ChromaDB
              retrieval/
                    retriever.py      # Recherche semantique
              generation/
                    llm.py            # Interface Ollama
              evaluation/
                  ragas_eval.py       # Evaluation du pipeline
        data/
              documents/              # Documents a indexer
              test_dataset/           # Dataset d'evaluation
        tests/
            test_parser.py
            test_chunker.py
            test_retriever.py
```

# Chapitre 6

# Évaluation et Métriques

## 6.1 Framework d'évaluation RAGAS

L'évaluation du système RAG sera réalisée avec le framework **RAGAS**, qui mesure objectivement les performances selon quatre dimensions complémentaires.

| Métrique | Ce qu'elle mesure | Cible |
|---|---|---|
| **Context Recall** | Les chunks pertinents sont-ils tous retrouvés ? | $\geq 0.75$ |
| **Context Precision** | Y a-t-il du bruit dans les chunks récupérés ? | $\geq 0.70$ |
| **Faithfulness** | La réponse est-elle fidèle aux sources (sans hallucination) ? | $\geq 0.80$ |
| **Answer Relevancy** | La réponse répond-elle à la question posée ? | $\geq 0.75$ |

TABLE 6.1 – Métriques RAGAS et objectifs cibles

## 6.2 Dataset de test

— **Composition** : 30 paires (question, réponse de référence) couvrant les 3 types de sources

— **Répartition** : 15 questions sur PDF, 10 sur données SQL, 5 sur texte libre

— **Construction** : manuelle par l'étudiant dès la semaine S1 sur le corpus de test

— **Format** : fichier JSON structuré compatible avec RAGAS

# Chapitre 7

# Risques et Mitigations

| Risque | Probabilité | Impact | Mitigation |
|---|---|---|---|
| Machine insuffisante pour Mistral 7B | Moyenne | Élevé | Utiliser `phi3:mini` (4 GB RAM) comme fallback |
| PDF scannés de mauvaise qualité | Élevée | Moyen | Prétraitement image (contraste, deskew) avant OCR |
| Chunking inadapté → mauvais retrieval | Moyenne | Élevé | Tests unitaires du chunker dès S3, ajustement itératif |
| Dérive de scope (feature creep) | Élevée | Élevé | Respecter strictement le périmètre du CDC, MoSCoW |
| Manque de temps pour Docker | Faible | Moyen | Dockeriser progressivement dès S4, pas en une fois |

TABLE 7.1 – Matrice des risques du projet

# Chapitre 8

# Livrables Attendus

**L1.** **Code source** versionné sur Git avec README complet

**L2.** **Docker Compose** permettant de démarrer le système en une commande

**L3.** **Rapport technique** documentant l'architecture, les choix techniques et l'état de l'art

**L4.** **Rapport d'évaluation** avec métriques RAGAS sur le dataset de test

**L5.** **Présentation** de soutenance (15–20 slides) avec démonstration live

**L6.** **Ce cahier des charges** finalisé et signé

# Glossaire

**RAG** *Retrieval-Augmented Generation* — technique combinant recherche documentaire et génération de texte par LLM.

**LLM** *Large Language Model* — modèle de langage de grande taille (ex : Mistral, LLaMA).

**Embedding** Représentation vectorielle numérique d'un texte capturant son sens sémantique.

**Chunk** Fragment de texte issu du découpage d'un document, unité de base de l'indexation.

**Chunking** Processus de découpage d'un document en chunks.

**Vector Store** Base de données spécialisée pour le stockage et la recherche de vecteurs.

**Ollama** Outil open-source permettant d'exécuter des LLM localement.

**Retriever** Composant responsable de retrouver les chunks les plus pertinents pour une requête.

**RAGAS** Framework d'évaluation automatique des pipelines RAG.

**OCR** *Optical Character Recognition* — reconnaissance optique de caractères dans les images.

**Souveraineté** Garantie que les données restent exclusivement sous le contrôle de l'organisation.

**Similarité cosinus** Mesure de similarité entre deux vecteurs, utilisée pour le retrieval.

# Signatures et Validation

Ce cahier des charges a été lu, compris et approuvé par les parties suivantes :

| **Étudiant(e)** | **Encadrant(e)** |
|---|---|
| Nom : _____ | Nom : _____ |
| Signature : | Signature : |
| Date : _____ | Date : _____ |